

Pokhara University
Gandaki College of Engineering and Science

Lamachaur, Kaski



Lab 2: Implementation of Source Code Management Using Git
Commands Through GitHub – Build, Breakpoints, Refactoring,
Collaboration, Copilot, and IntelliSense

Submitted By:

Ujjwal Rana

Roll no. 43

Submitted To:

Er. Rajendra Bahadur Thapa

Lab Report

Title

Source Code Management Using Git, GitHub, and Modern IDE Development Features

Objective

To implement Source Code Management (SCM) using Git and GitHub while utilizing modern Integrated Development Environment (IDE) features such as Build, Breakpoints, Refactoring, Collaboration, GitHub Copilot, and IntelliSense to improve software development, debugging, and version control.

Tools Required

- Git
- GitHub Account
- Visual Studio Code or Visual Studio
- GitHub Copilot Extension
- Git Extension
- Sample Project (C++, Java, Python, or C#)

Theory

Source Code Management (SCM) is the practice of tracking, organizing, and managing changes made to source code throughout the software development lifecycle. It enables developers to maintain version history, collaborate efficiently, and recover previous versions when necessary.

Git is a distributed version control system that allows developers to manage source code locally while maintaining a complete history of project changes. GitHub is a cloud-based platform that hosts Git repositories and provides features for collaboration, code sharing, version tracking, pull requests, and project management.

Modern IDEs further enhance software development by providing productivity tools such as Build for compiling applications, Breakpoints for debugging, Refactoring for improving code structure, IntelliSense for intelligent code completion and error detection, and GitHub Copilot for AI-assisted code generation and development support.

Procedure

1. Install Git and configure the global username and email address.
2. Create a new project directory and initialize it as a local Git repository using the `git init` command.
3. Develop a sample application and add the project files to the repository.
4. Stage the project files and commit the changes to the local repository.

5. Create a new repository on GitHub and connect it to the local repository.
6. Push the local project to the remote GitHub repository.
7. Build the application and verify that it compiles successfully without errors.
8. Set breakpoints and debug the program to inspect variables and program execution.
9. Perform code refactoring to improve readability, maintainability, and code organization without changing functionality.
10. Use IntelliSense to assist with code completion, syntax checking, and parameter suggestions.
11. Utilize GitHub Copilot to generate code suggestions and accelerate development.
12. Create a feature branch, implement changes, commit the updates, merge the branch into the main branch, and synchronize the repository with GitHub.

Git Commands (Coding Section)

Configure Git

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your@email.com"
```

Initialize repository

```
git init
```

Add project files

```
git add .
```

Commit changes

```
git commit -m "Initial project commit"
```

Rename default branch

```
git branch -M main
```

Connect to GitHub repository

```
git remote add origin https://github.com/username/project.git
```

Push repository

```
git push -u origin main
```

Create a feature branch

```
git checkout -b feature-update
```

Stage and commit branch changes

```
git add .
```

```
git commit -m "Added new feature"
```

Merge feature branch

```
git checkout main
```

```
git merge feature-update
```

Check repository status

```
git status
```

View commit history

```
git log
```

IDE Features Used

Build

The project was compiled successfully using the IDE's build system. The application executed without compilation errors, confirming that the source code was syntactically correct.

Breakpoints

Breakpoints were placed at specific lines of code to pause program execution during debugging. Variable values, execution flow, and logical behavior were inspected to identify and resolve potential issues.

Refactoring

The source code was refactored by renaming variables and functions, simplifying repetitive code, improving formatting, and enhancing readability while preserving the original functionality.

Collaboration

Git branches were created to isolate feature development. Changes were committed, merged into the main branch, and synchronized with GitHub, demonstrating an effective collaborative development workflow.

GitHub Copilot

GitHub Copilot was used to generate intelligent code suggestions, assist in writing functions, and improve development efficiency by reducing repetitive coding tasks.

IntelliSense

IntelliSense provided automatic code completion, syntax highlighting, parameter information, and real-time error detection, resulting in faster and more accurate code development.

Output

- Git was successfully configured with the user's credentials.
- A local Git repository was initialized successfully.
- The sample project was added and committed to the repository.
- The local repository was successfully connected to GitHub.
- The project was pushed to the remote repository without errors.
- The application was built and executed successfully.
- Breakpoints were used to debug the application and inspect program execution.
- Code was successfully refactored without affecting functionality.
- Git branches were created, merged, and synchronized with GitHub.
- GitHub Copilot generated useful code suggestions during development.
- IntelliSense assisted with code completion, syntax checking, and error detection.

Result

The experiment successfully implemented Source Code Management using Git and GitHub while demonstrating the effective use of modern IDE features, including Build, Breakpoints, Refactoring, Collaboration, GitHub Copilot, and IntelliSense. These tools improved software development efficiency, code quality, debugging capabilities, and version control practices.

Conclusion

This experiment provided practical experience with Source Code Management using Git and GitHub and demonstrated the importance of modern IDE features in software development. Version control enabled efficient tracking and management of source code changes, while Build, Breakpoints, Refactoring, Collaboration, GitHub Copilot, and IntelliSense improved productivity, debugging, code maintainability, and teamwork. Together, these tools form an essential workflow for modern software engineering and Agile development.

